

Deep Learning Course Project

Fashion-MNIST Image Classification with CNN

Faculty of Computers and Information

May 15, 2026

Contents

1	Problem Description	2
2	Dataset	2
3	Model Architectures	2
3.1	Model 1 — Basic CNN + SGD	2
3.2	Model 2 — Enhanced CNN + Adam	3
4	Training Setup	3
5	Results	4
6	Project Structure	4
7	Instructions for Running the Project	4
7.1	Prerequisites	4
7.2	Steps	5
8	Dataset Link	5

1 Problem Description

The objective of this project is to classify grayscale images of clothing items into **10 distinct categories** using Convolutional Neural Networks (CNNs).

Two model architectures are compared to study the effect of:

- Optimizer choice (SGD vs. Adam)
- Network depth (1 vs. 2 convolutional blocks)
- Regularization techniques (Dropout, Batch Normalization)
- Learning rate scheduling

Target Classes

T-shirt/top, Trouser, Pullover, Dress, Coat,
Sandal, Shirt, Sneaker, Bag, Ankle boot

2 Dataset

- **Name:** Fashion-MNIST
- **Source:** <https://github.com/zalandoresearch/fashion-mnist>
- **Total images:** 70,000 grayscale images (28×28 pixels)
 - Training set: 60,000 images
 - Test set: 10,000 images
- **Number of classes:** 10
- **Download:** Automatically via `torchvision.datasets.FashionMNIST`

Preprocessing

- Conversion to tensor via `transforms.ToTensor()`
- Normalization: $\text{mean} = 0.5$, $\text{std} = 0.5$
- Batch size: 64

3 Model Architectures

3.1 Model 1 — Basic CNN + SGD

- **Optimizer:** SGD with learning rate = 0.001
- **Loss function:** `CrossEntropyLoss`
- **Epochs:** 15

Table 1: Model 1 Architecture

Layer	Type	Details
1	Conv2d	$1 \rightarrow 16$ channels, kernel = 3, padding = 1
2	ReLU	Activation function
3	MaxPool2d	kernel = 2, stride = 2
4	Flatten	Output: $16 \times 14 \times 14 = 3136$
5	Linear	$3136 \rightarrow 64$
6	ReLU	Activation function
7	Linear	$64 \rightarrow 10$ (output)

3.2 Model 2 — Enhanced CNN + Adam

Table 2: Model 2 Architecture

Layer	Type	Details
1	Conv2d	$1 \rightarrow 32$ channels, kernel = 3, padding = 1
2	BatchNorm2d	32 channels
3	ReLU	Activation function
4	MaxPool2d	kernel = 2, stride = 2
5	Conv2d	$32 \rightarrow 64$ channels, kernel = 3, padding = 1
6	BatchNorm2d	64 channels
7	ReLU	Activation function
8	MaxPool2d	kernel = 2, stride = 2
9	Flatten	Output: $64 \times 7 \times 7 = 3136$
10	Linear	$3136 \rightarrow 128$
11	ReLU	Activation function
12	Dropout	$p = 0.5$
13	Linear	$128 \rightarrow 10$ (output)

- **Optimizer:** Adam with learning rate = 0.01
- **Scheduler:** StepLR (step_size = 5, $\gamma = 0.5$)
- **Loss function:** CrossEntropyLoss
- **Epochs:** 15

4 Training Setup

Both models were trained for 15 epochs. At each epoch the following metrics were tracked:

- Training Loss and Training Accuracy

- Validation Loss and Validation Accuracy

Training and validation curves (Loss and Accuracy vs. Epochs) were plotted for both models to visualize learning progress and detect overfitting.

5 Results

Table 3: Final Results Comparison

Model	Optimizer	Enhancements	Val Accuracy	Val Loss
Model 1 (Basic CNN)	SGD	None	~82%	~0.15
Model 2 (Enhanced CNN)	Adam + StepLR	BatchNorm, Dropout, 2 Conv	~91%	~0.05

Key Findings

- Model 2 significantly outperforms Model 1 in both accuracy and loss.
- **Batch Normalization** stabilizes training and accelerates convergence.
- **Dropout** ($p = 0.5$) reduces overfitting effectively.
- **Adam** with learning rate scheduling converges faster than plain SGD.
- Increasing depth from 1 to 2 convolutional blocks improves feature extraction.

6 Project Structure

Listing 1: Repository Layout

```

1 .
2     project_DeepLearning.ipynb    # Main notebook (all code)
3     README.md                    # Project overview
4     data/                         # Auto-downloaded by
    torchvision

```

7 Instructions for Running the Project

7.1 Prerequisites

Install the required Python packages:

Listing 2: Install Dependencies

```

1 pip install torch torchvision matplotlib pandas numpy

```

7.2 Steps

1. Clone the repository:

```
1 git clone https://github.com/<your-username>/<repo-name>.git
2 cd <repo-name>
```

2. Install dependencies:

```
1 pip install torch torchvision matplotlib pandas numpy
```

3. Launch Jupyter Notebook:

```
1 jupyter notebook project_DeepLearning.ipynb
```

4. Run all cells in order.

The Fashion-MNIST dataset will be downloaded automatically on the first run.

Note

For free GPU access, upload the notebook to **Google Colab** and enable GPU runtime via *Runtime* → *Change runtime type* → *T4 GPU*.

8 Dataset Link

<https://github.com/zalandoresearch/fashion-mnist>

Available via torchvision:

```
1 from torchvision import datasets
2 train_set = datasets.FashionMNIST(root='./data', train=True,
  download=True, ...)
3 test_set  = datasets.FashionMNIST(root='./data', train=False,
  download=True, ...)
```